

Міністерство освіти і науки України
Прикарпатський національний університет
імені Василя Стефаника
Факультет математики та інформатики
Кафедра інформаційних технологій

Програмування ігрових застосунків

ЗВІТ

про виконання завдання

**Тема: Розробка 2D-гри жанру Tower Defense
на платформі Unity з розгортанням у WebGL**

Виконав: студент групи ІПЗ-31

Гнатишак А. Й.

Викладач: Ровінський В. А.

ЗМІСТ

1 Постановка задачі.....	3
2 Опис правил та геймплею	3
3 Схема станів та ключові скрипти	5
4 Структура префабів і ScriptableObject-даних.....	7
5 Економіка та формування хвиль	8
6 Object Pooling та оптимізація для WebGL	8
7 Перелік ассетів	10
Висновки.....	11

1 ПОСТАНОВКА ЗАДАЧІ

Розробити двовимірну гру жанру Tower Defense на ігровому рушії Unity з цільовою платформою WebGL та публікацією на GitHub Pages. Гравець у ролі Захисника будує оборонні вежі на сітці 12×8 , а автоматичний AI-противник формує хвилі ворогів. Мета — встояти 10 раундів.

Перелік обов'язкових завдань:

- сітка 12×8 і фіксований Z-подібний шлях через 6 точок-маршрутів;
- 4 типи веж і 3 типи ворогів з відмінними характеристиками;
- машина станів з 5 фазами і AI-формування хвиль на 10 раундів;
- Object Pooling для ворогів і снарядів — підтримка 60 fps при 20 вежах і 50 ворогах одночасно;
- процедурна генерація спрайтів і звуків без зовнішніх ассетів;
- складання WebGL-білда і публікація онлайн.

2 ОПИС ПРАВИЛ ТА ГЕЙМПЛЕЮ

2.1 Основні правила

Гравець — Захисник. Будує вежі на полі, аби зупиняти ворожі хвилі. База має 20 HP; кожен ворог, що дійде до бази, зменшує HP на 1. Програш — HP=0. Перемога — пережити всі 10 раундів. Опонент — AI-Атакуючий: у фазі підготовки формує хвилю в межах бюджету (стартовий 200, +50 щораунд).

2.2 Цикл раунду

- Preparation — гравець купує і ставить вежі.
- Battle — WaveManager спавнить ворогів з інтервалом 0,8–1,2 с; вежі стріляють автоматично.
- RoundEnd — підрахунок винагороди, +50 до бюджету AI.
- Перехід до наступного раунду або GameOver.

2.3 Характеристики веж

Вежа	Ціна (g)	Швидкість	Шкода	Дальність	Тип атаки
Лучник	100	1,0 / с	20	3,0	Одна ціль
Маг	150	0,6 / с	30	2,0	АоЕ r=1,5
Льодяник	120	1,0 / с	10	3,0	Slow –50% / 2с
Гармата	200	0,3 / с	65	4,5	Одна ціль

2.4 Характеристики ворогів

Ворог	НР	Швидкість	Бюджет АІ	Винагород а	Особливість
Гоблін	40	3,0	10	5 g	—
Орк	150	1,5	25	12 g	висока стійкість
Привид	80	2,0	20	8 g	імунітет до Slow

2.5 Керування

Дія	Кнопка / спосіб
Обрати вежу для встановлення	ЛКМ по кнопці у бічній панелі
Поставити обрану вежу	ЛКМ по вільній (зеленій) клітинці
Показати радіус дії поставленої вежі	ЛКМ по вежі
Скасувати вибір / сховати радіус	ПКМ
Розпочати фазу битви	Кнопка «ПОЧАТИ ХВИЛЮ» у HUD
Рестарт після GameOver	Кнопка «ГРАТИ ЗНОВУ»

3 СХЕМА СТАНІВ ТА КЛЮЧОВІ СКРИПТИ

3.1 Машина станів

Центральне архітектурне рішення — машина станів (State Machine) з 5 фазами і подією OnStateChanged, на яку підписуються всі підсистеми (патерн «Спостерігач»).

Перелік станів і переходів:

Стан	Опис	Перехід
Menu	Головне меню	ГРАТИ → Preparation
Preparation	Купівля та встановлення веж	ПОЧАТИ ХВИЛЮ → Battle
Battle	Спавн ворогів, стрільба	усі знищені → RoundEnd
RoundEnd	Підрахунок раунду	є раунди → Preparation
GameOver	Перемога / Поразка	ГРАТИ ЗНОВУ → Menu

Реалізація центрального компонента:

```
public enum GameState { Menu, Preparation, Battle, RoundEnd, GameOver
}

public class GameStateManager : MonoBehaviour {
    public static GameStateManager Instance { get; private set; }
    public GameState CurrentState { get; private set; }
    public event Action<GameState> OnStateChanged;
    public void SetState(GameState s) {
        CurrentState = s;
        OnStateChanged?.Invoke(s);
    }
}
```

3.2 Ключові скрипти

Скрипт	Призначення
GameStateManager	Машина станів з подією OnStateChanged
GameManager	Загальна координація, ініціалізація, рестарт
GridManager	Сітка 12×8, перевірка CanBuild
WaveManager	Корутина спавну ворогів
AIAttacker	Формування складу хвилі за бюджетом
EconomyManager	Золото Захисника, бюджет AI, події змін

Base	НР бази, подія OnBaseDestroyed
EnemyPool	Об'єктний пул ворогів (ObjectPool<T>)
ProjectilePool	Об'єктний пул снарядів
TowerPlacer	Обробка кліків, розміщення веж, радіус
TowerShooter	Таргетинг, кулдаун, спавн снаряда
Projectile	Політ до цілі, шкода (Single/AoE/Slow)
EnemyMover	Рух waypoint→waypoint
EnemyHealth	НР та індикатор над ворогом
HudUI / TowerSidebar / GameOverUI	Інтерфейс гри
AudioBus	Процедурна генерація звуків
SceneBootstrapper	Створення спрайтів і префабів

4 СТРУКТУРА ПРЕФАБІВ І SCRIPTABLEOBJECT-ДАНИХ

4.1 Префаб ворога

```
Enemy_<Type>                                ← root, layer "Enemy"  
├─ SpriteRenderer (sortingOrder=3)  
├─ CircleCollider2D (r=0,42, trigger)  
├─ [Enemy] [EnemyMover] [EnemyHealth]  
└─ HPCanvas (WorldSpace, sortingOrder=10)  
    └─ Frame, BG, Fill (Image — anchorMax.x = ratio)
```

HP-індикатор реалізовано через зміну `anchorMax.x` на `Image` (надійніше за `UISlider` у `WorldSpace`). Атрибут `[DisallowMultipleComponent]` на `EnemyHealth` і `EnemyMover` запобігає дублюванню компонентів через `RequireComponent`.

4.2 Префаб вежі та снаряда

```
Tower_<Type>                                Proj_<Type>  
├─ SpriteRenderer (sortingOrder=2) └─ SpriteRenderer  
(sortingOrder=4)  
├─ BoxCollider2D (0,8×0,8, trigger) └─ [Projectile]  
└─ [Tower] [TowerShooter]
```

4.3 ScriptableObject TowerData

`ScriptableObject` — асет-файл, що описує конфігурацію вежі. Перевага: дизайнер міняє баланс в `Inspector` без перекомпіляції; усі екземпляри одного типу посилаються на один `SO` — економія пам'яті.

```
[CreateAssetMenu]  
public class TowerData : ScriptableObject {  
    public string towerName;  
    public int    cost;  
    public float  fireRate, range, damage;  
    public AttackType attackType; // Single, AoE, Slow  
    public float  slowFraction, aoeRadius, slowDuration;  
    public GameObject towerPrefab, projectilePrefab;  
}
```

Створено 4 інстанси: `ArcherData`, `MageData`, `FreezerData`, `CannonData`.

5 ЕКОНОМІКА ТА ФОРМУВАННЯ ХВИЛЬ

5.1 Економічні параметри

Параметр	Значення	Коментар
----------	----------	----------

Стартове золото	300 g	капітал Захисника
Стартовий бюджет AI	200 g	на першу хвилю
Приріст бюджету AI	+50 g/раунд	лінійне зростання складності
НР бази	20	-1 за кожного ворога
Кількість раундів	10	фіксована
Макс. ворогів на полі	50	обмеження WaveManager

5.2 Алгоритм формування хвилі

Метод `AIAttacker.BuildWave` заповнює чергу ворогів випадковим вибором у межах бюджету. Зростання складності реалізовано через імовірнісне перемикання типів ворогів:

```
public Queue<EnemyData> BuildWave(int round, int budget) {
    var queue = new Queue<EnemyData>();
    int spent = 0;
    while (spent + goblinData.cost <= budget) {
        EnemyData choice = goblinData;
        float r = UnityEngine.Random.value;
        if (round >= 5 && r < 0.25f) choice = ghostData;
        else if (round >= 3 && r < 0.55f) choice = orcData;
        if (spent + choice.cost > budget) choice = goblinData;
        queue.Enqueue(choice);
        spent += choice.cost;
    }
    return queue;
}
```

Раунди	Склад хвилі
1–2	лише Гобліни (тренування)
3–4	Гобліни 45% + Орки 55% (потрібна Гармата)
5–10	Гобліни + Орки + Привиди 25% (комплексний склад)

6 OBJECT POOLING ТА ОПТИМІЗАЦІЯ ДЛЯ WebGL

Виклики `Instantiate/Destroy` спричиняють алокацію та роботу `GarbageCollector`, що у `WebGL` призводить до фризів. Тому використано об'єктний пул `UnityEngine.Pool.ObjectPool<T>`: «знищення» = `SetActive(false)` + повернення у пул, «створення» = взяття з пулу + `SetActive(true)`.

```
_pool[data] = new ObjectPool<Enemy>()
```



```

createFunc:      () =>
Instantiate(data.prefab).GetComponent<Enemy>(),
actionOnGet:     e => e.gameObject.SetActive(true),
actionOnRelease: e => e.gameObject.SetActive(false),
actionOnDestroy: e => Destroy(e.gameObject),
defaultCapacity: 15, maxSize: 50);

```

Pre-warm на старті (15 Гоблінів, 8 Орків, 8 Привидів) усуває фриз при першому спавні.

Оптимізація	Ефект
IL2CPP scripting backend	нативна швидкість vs Mono
compressionFormat = Disabled	сумісність з GitHub Pages
exceptionSupport = None	–20% розміру білда
memorySize = 256 MB	стельова межа для 50 ворогів + UI
Процедурні спрайти/аудіо	відсутність зовнішніх ассетів
DontDestroyOnLoad на префабах	одноразова ініціалізація
raycastTarget = false на декорі	–overhead GraphicRaycaster
[DisallowMultipleComponent]	захист від дублікатів

Фінальний розмір білда — 48 МБ (WASM 38 МБ + asset data 9,2 МБ + framework та loader 0,5 МБ). Холодне завантаження у браузері — 5–10 с. Стабільні 60 fps при 20 вежах і 50 ворогах одночасно.

7 ПЕРЕЛІК АССЕТІВ

7.1 Процедурні спрайти (Texture2D.SetPixels)

Сутність	Розмір	Стиль
Лучник	20×30	піксельна вежа з прапором
Маг	20×30	фіолетова з зірочкою
Льодяник	20×30	блакитна з кристалом
Гармата	20×30	темна з гарматою
Гоблін	14×16	зелений людиноподібний
Орк	16×18	темно-зелений з обладунком
Привид	14×14	напівпрозорий блакитний
Стріла / Куля / Кристал / Ядро	6–12 px	спрайти снарядів
Замок (база)	40×32	три вежі з брамою
Трава / Камінь шляху	8×8	тайли поля

7.2 Процедурне аудіо (AudioClip.Create)

Звук	Спосіб синтезу	Тривалість
Постріл вежі	sin 880 Hz + decay	0,05 с
Попадання	білий шум + decay	0,08 с
Вибух (AoE)	шум + sin 80 Hz	0,30 с
Заморозка	sweep 1400→300 Hz	0,18 с
Смерть ворога	sweep 500→80 Hz	0,30 с
Підбір золота	sin 1320 Hz	0,10 с
Удар по базі	білий шум	0,20 с
Перемога / Поразка	арпеджіо	0,5–0,7 с

ВИСНОВКИ

Розроблено повноцінну 2D гру жанру Tower Defense на Unity 6 з публікацією у браузері через GitHub Pages (<https://modertok.github.io/towerDefenseOld/>). Реалізовано всі завдання: машина станів, 4 типи веж і 3 типи ворогів зі ScriptableObject-конфігураціями, AI-формування хвиль на 10 раундів, об'єктні пули, процедурні спрайти і звуки, WebGL-білд (48 МБ). Продуктивність 60 fps стабільна при 20 вежах і 50 ворогах одночасно. Проєкт демонструє практичне застосування шаблонів проєктування State Machine, Observer та Object Pool, а також особливостей розробки під веб.